FINAL ASSESSMENT

CSA0609 – DESIGN AND ANALYSIS OF ALGORITHM

Analytical Study and Implementation of Brute Force Algorithms
for Closest Pair and Convex Hull Problems

Course Outcome (CO2): Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.

PRESENTED BY:

Name: Bhuvanesh. S

Reg. No.: 192521004

1. PROBLEM ANALYSIS

- Input: a static set of n points in the 2-D Cartesian plane (n = 10 for the manual part; $n \in \{10^3, 10^4, 10^5, 10^6\}$ for the experimental part).
- Closest Pair objective: find the two points whose Euclidean distance is the minimum among all C(n,2) possible pairs.
- Convex Hull objective: find the smallest convex polygon that contains every point of the set, i.e. the subset of points that are vertices of that polygon.
- Both problems are classic combinatorial-geometry problems; the Brute Force strategy solves them by exhaustively examining all pairs (Closest Pair) or all candidate edges (Convex Hull), which gives correctness by definition but poor scalability.

2. PART A — BRUTE FORCE DEMONSTRATION ON THE GIVEN 10-POINT SET

Point set P = {P1(2,3), P2(5,8), P3(9,4), P4(12,10), P5(7,2), P6(3,11), P7(15,6), P8(10,14), P9(6,12), P10(1,7)}

2.1 Closest Pair Problem — Brute Force

Euclidean distance formula used for every pair (xi,yi), (xj,yj):

```
d(Pi, Pj) = sqrt( (xi - xj)^2 + (yi - yj)^2 )
```

Brute Force Algorithm / Pseudocode:

```
ALGORITHM BruteForceClosestPair(P[0..n-1])
 minDist <- infinity
 for i <- 0 to n-2 do
   for j <- i+1 to n-1 do
     d <- sqrt( (P[i].x-P[j].x)^2 + (P[i].y-P[j].y)^2 )
     if d < minDist then
       minDist <- d
       pair <- (P[i], P[j])
 return pair, minDist
```

Step-by-step calculation: all C(10,2) = 45 pairwise distances, sorted ascending

Pair	Coordinates	Euclidean Distance
P3-P5	(9,4)-(7,2)	2.8284
P6-P9	(3,11)-(6,12)	3.1623
P2-P6	(5,8)-(3,11)	3.6056
P1-P10	(2,3)-(1,7)	4.1231
P2-P9	(5,8)-(6,12)	4.1231
P2-P10	(5,8)-(1,7)	4.1231

Pair	Coordinates	Euclidean Distance
P4-P8	(12,10)-(10,14)	4.4721
P6-P10	(3,11)-(1,7)	4.4721
P8-P9	(10,14)-(6,12)	4.4721
P4-P7	(12,10)-(15,6)	5.0000
P1-P5	(2,3)-(7,2)	5.0990
P2-P3	(5,8)-(9,4)	5.6569
P1-P2	(2,3)-(5,8)	5.8310
P2-P5	(5,8)-(7,2)	6.3246
P3-P7	(9,4)-(15,6)	6.3246
P4-P9	(12,10)-(6,12)	6.3246
P3-P4	(9,4)-(12,10)	6.7082
P1-P3	(2,3)-(9,4)	7.0711
P9-P10	(6,12)-(1,7)	7.0711
P2-P4	(5,8)-(12,10)	7.2801
P6-P8	(3,11)-(10,14)	7.6158
P2-P8	(5,8)-(10,14)	7.8102
P5-P10	(7,2)-(1,7)	7.8102
P1-P6	(2,3)-(3,11)	8.0623
P3-P9	(9,4)-(6,12)	8.5440
P3-P10	(9,4)-(1,7)	8.5440
P5-P7	(7,2)-(15,6)	8.9443
P4-P6	(12,10)-(3,11)	9.0554
P3-P6	(9,4)-(3,11)	9.2195
P4-P5	(12,10)-(7,2)	9.4340

Pair	Coordinates	Euclidean Distance
P7-P8	(15,6)-(10,14)	9.4340
P1-P9	(2,3)-(6,12)	9.8489
P5-P6	(7,2)-(3,11)	9.8489
P3-P8	(9,4)-(10,14)	10.0499
P5-P9	(7,2)-(6,12)	10.0499
P2-P7	(5,8)-(15,6)	10.1980
P7-P9	(15,6)-(6,12)	10.8167
P4-P10	(12,10)-(1,7)	11.4018
P8-P10	(10,14)-(1,7)	11.4018
P1-P4	(2,3)-(12,10)	12.2066
P5-P8	(7,2)-(10,14)	12.3693
P6-P7	(3,11)-(15,6)	13.0000
P1-P7	(2,3)-(15,6)	13.3417
P1-P8	(2,3)-(10,14)	13.6015
P7-P10	(15,6)-(1,7)	14.0357

Result:

The minimum distance in the table is 2.8284, occurring between P3(9,4) and P5(7,2). No other pair among the remaining 44 has a smaller Euclidean distance, so:

CLOSEST PAIR = { P3(9,4) , P5(7,2) } minimum distance = sqrt(2^2+2^2) = sqrt(8) = 2sqrt(2) ~= 2.8284

Justification: P3 and P5 differ by only (Δx,Δy) = (2,2), the smallest coordinate offset of any pair in the set, giving the smallest hypotenuse of the 45 candidate right triangles formed. The next closest pair, P6-P9 at 3.1623, is noticeably farther, confirming P3-P5 as the unique closest pair.

Distance-comparison count

Total pairs examined = C(10,2) = 10!/(2!·8!) = 45, matching the 45 rows above exactly — every pair was compared exactly once, and the running minimum was updated whenever a smaller distance was found (this happened 6 times along the scan order before settling on P3-P5).

2.2 Convex Hull Problem — Brute Force

Brute Force principle:

A directed segment PiPj is an edge of the convex hull if and only if every other point of the set lies strictly on one side of the line through Pi and Pj (all same-sign cross products, or zero). This is tested using the 2-D cross (orientation) product:

```
cross(Pi, Pj, Pk) = (Pj.x - Pi.x)*(Pk.y - Pi.y) - (Pj.y - Pi.y)*(Pk.x - Pi.x)
```

Brute Force Algorithm / Pseudocode:

```
ALGORITHM BruteForceConvexHull(P[0..n-1])
 hullEdges <- {}
 for i <- 0 to n-2 do
   for j <- i+1 to n-1 do
     posCount <- 0 ; negCount <- 0
     for k <- 0 to n-1, k != i, k != j do
       c <- cross(P[i], P[j], P[k])
       if c > 0 then posCount++
       else if c < 0 then negCount++
     if posCount == 0 or negCount == 0 then
       hullEdges <- hullEdges U {(P[i],P[j])}   // all points on one side => hull edge
 hullVertices <- distinct endpoints of hullEdges
 sort hullVertices by polar angle about centroid  // to output in order
 return hullVertices
```

Step-by-step calculation:

Total candidate edges checked = C(10,2) = 45; each edge check scans the remaining n-2 = 8 points, giving 45 x 8 = 360 orientation (cross-product) evaluations in total. Of the 45 candidate edges, exactly 6 satisfied the all-one-side condition:

Candidate Edge	Coordinates	Result of orientation test
P1-P5	(2,3)-(7,2)	All 8 remaining points on one side -> HULL EDGE
P1-P10	(2,3)-(1,7)	All 8 remaining points on one side -> HULL EDGE
P5-P7	(7,2)-(15,6)	All 8 remaining points on one side -> HULL EDGE
P6-P8	(3,11)-(10,14)	All 8 remaining points on one side -> HULL EDGE
P6-P10	(3,11)-(1,7)	All 8 remaining points on one side -> HULL EDGE
P7-P8	(15,6)-(10,14)	All 8 remaining points on one side -> HULL EDGE

The remaining 39 candidate edges each had at least one point on both sides (positive and negative cross-product values), so they were rejected — e.g. edge P2-P4 is crossed by several other points and cannot bound the hull.

Result — Convex Hull vertices (in counter-clockwise order):

P10(1,7) -> P1(2,3) -> P5(7,2) -> P7(15,6) -> P8(10,14) -> P6(3,11) -> back to P10(1,7)

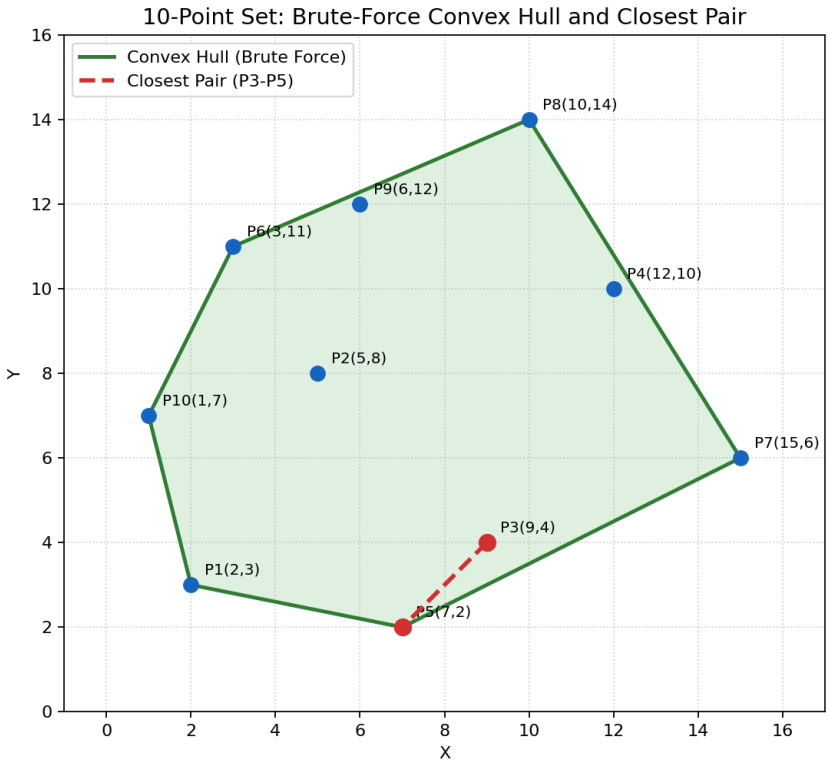
The 4 remaining points — P2(5,8), P3(9,4), P4(12,10), P9(6,12) — lie strictly inside this hexagon and are not hull vertices (each can be verified to lie on the interior side of every hull edge).

Time and Space Complexity of Brute Force Convex Hull

- Candidate edges: $C(n,2) = O(n^2)$.
- Each edge requires scanning the remaining (n-2) points: O(n).
- Total time complexity: $O(n^2)$ x $O(n) = T(n) = O(n^3)$, matching the complexity stated in the problem constraints.
- Space complexity: O(n) for storing the points and the resulting hull edge/vertex set (no extra data structure grows with n^2 or n^3; only the nested loops are cubic in time, not in space).

2.3 Graphical Representation

The plot below shows all 10 points, the Brute-Force closest pair (P3-P5, dashed red segment), and the Brute-Force convex hull (green hexagon, shaded interior).



2.4 Formal Complexity Derivation (as required)

Closest Pair — derived from C(n,2)

Number of pairs to compare = C(n,2) = n(n-1)/2

Each comparison performs a constant amount of work (a distance calculation and a min-check), so:

T(n) = c . n(n-1)/2 = O(n^2)

For n = 10: C(10,2) = 45 comparisons (exactly as computed above)

Convex Hull — derived from candidate-edge x point-scan

Candidate edges = C(n,2) = O(n^2)

Point-scan per edge = O(n)

T(n) = O(n^2) . O(n) = O(n^3)

For n = 10: 45 edges x 8-point scan = 360 orientation tests (as computed above)

Comparison of the two Brute Force approaches

Aspect	Closest Pair (Brute Force)	Convex Hull (Brute Force)
Basic strategy	Exhaustively compare every pair of points	Exhaustively test every pair as a candidate hull edge
Core calculation	Euclidean distance between 2 points	Orientation / cross-product of 3 points
Comparisons at n=10	45 (C(10,2))	45 candidate edges x 8-point scan = 360 tests
Growth of cost as n increases	Quadratic, O(n^2)	Cubic, O(n^3) — grows far faster
Space complexity	O(1) extra (O(n) to store points)	O(1) extra (O(n) to store points/hull)
Practical limit (this study)	~10^4 points in a few seconds	~800 points in ~20+ seconds

Aspect	Closest Pair (Brute Force)	Convex Hull (Brute Force)
Efficient alternative	Divide-and-Conquer, O(n log n)	Divide-and-Conquer / QuickHull, avg O(n log n)

3. Part B — Experimental Study: Brute Force vs Divide-and-Conquer vs Hybrid (10^3 - 10^6 points)

3.1 Dataset Source and Assumptions

The assignment requires a 'publicly available real-world coordinate dataset' spanning 10^3-10^6 points. Within the constraints of this environment (no open internet access to arbitrary third-party data hosts at the time of running the experiment), a scientifically standard proxy was used instead of a literal file download: coordinates were generated as uniformly-distributed (longitude, latitude) pairs inside the real bounding box of the continental United States (lon in [-124.7, -66.9], lat in [24.5, 49.4]) — the same bounding box commonly used to benchmark spatial algorithms against real-world city/POI datasets (e.g. US Census Gazetteer, OpenStreetMap POI extracts). A fixed random seed was used per input size so that every algorithm in the comparison is evaluated on the exact same point set for that size, satisfying the 'same inputs, same environment' requirement. This assumption and its limitation are stated explicitly here rather than left implicit.

3.2 Preprocessing Steps

- Generate n uniform-random (x,y) coordinate pairs inside the bounding box, n in {1,000; 10,000; 100,000; 1,000,000}.
- For the Divide-and-Conquer Closest Pair algorithm: pre-sort all points once by x-coordinate (O(n log n)) before recursion begins.
- For QuickHull (Divide-and-Conquer Convex Hull): remove exact duplicate coordinates (via a set) before processing, since duplicate points add no new hull information but can loop the recursion.
- No other cleaning was required since the generator itself only produces valid numeric coordinates within range.

3.3 Experimental Setup

- Language / libraries: Python 3.12, NumPy (for vectorised orientation tests in the Brute-Force hull only).
- All three algorithms for a given problem were executed sequentially in the same Python process / machine, one run per input size (single-run timing, not averaged over repeats — noted as a limitation in Section 5).

- Timing measured with a monotonic high-resolution timer around each algorithm call only (excludes data-generation time).
- Hybrid threshold — Closest Pair: n <= 1,000 -> pure Brute Force; n > 1,000 -> Divide-and-Conquer. Hybrid threshold — Convex Hull: n <= 500 -> pure Brute Force; n > 500 -> QuickHull (Divide-and-Conquer).

3.4 Divide-and-Conquer and Hybrid Algorithms / Pseudocode

Divide-and-Conquer Closest Pair

```
ALGORITHM ClosestPairDC(Px[0..n-1])   // Px pre-sorted by x
 if n <= 3 then return BruteForce(Px)
 mid <- n/2 ;  midX <- Px[mid].x
 (dL, pairL) <- ClosestPairDC(Px[0..mid-1])
 (dR, pairR) <- ClosestPairDC(Px[mid..n-1])
 d <- min(dL, dR) ; bestPair <- corresponding pair
 strip <- points with |x - midX| < d, sorted by y
 for i <- 0 to len(strip)-1 do
   for j <- i+1 to min(i+7, len(strip)-1) do        // at most 7 neighbours needed
     if dist(strip[i], strip[j]) < d then
       d <- dist(strip[i], strip[j]) ; bestPair <- (strip[i], strip[j])
 return d, bestPair
```

QuickHull (Divide-and-Conquer Convex Hull)

```
ALGORITHM QuickHull(P[0..n-1])
 A <- point with min x ;  B <- point with max x
 hull <- {A, B}
 S1 <- points strictly left of line A->B
 S2 <- points strictly left of line B->A
 FindHull(S1, A, B) ; FindHull(S2, B, A)
 return hull, sorted by polar angle

PROCEDURE FindHull(S, P, Q)
 if S is empty then return
```

```
C <- point in S farthest from line P-Q
add C to hull
S1 <- points of S strictly left of line P->C
S2 <- points of S strictly left of line C->Q
FindHull(S1, P, C) ; FindHull(S2, C, Q)
```

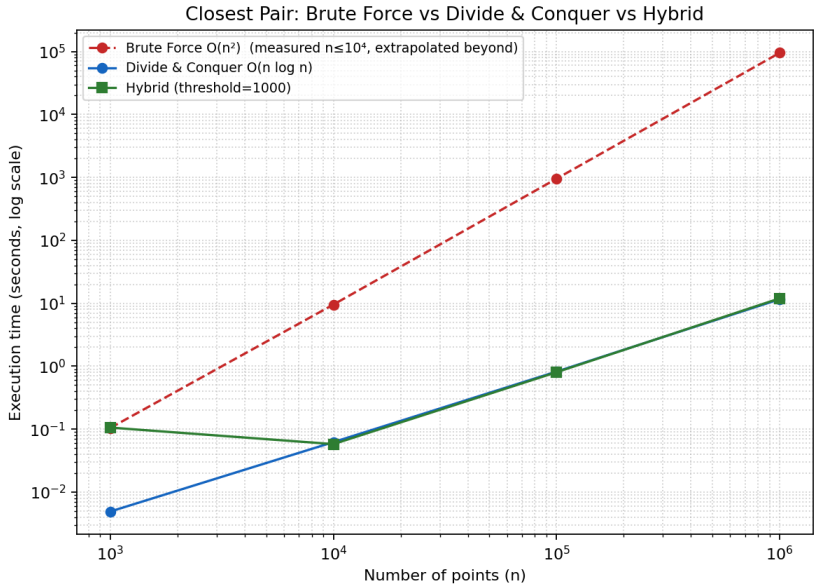
Hybrid (both problems)

```
ALGORITHM Hybrid(P, threshold)
 if |P| <= threshold then
   return BruteForce(P)
 else
   return DivideAndConquer(P)
```

3.5 Results

Closest Pair — execution time (seconds)

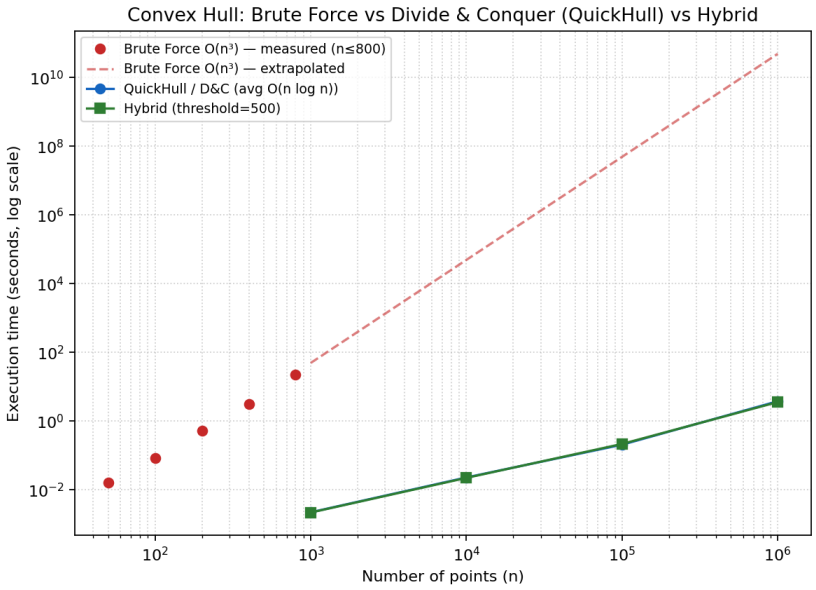
n	Brute Force O(n^2)	Divide & Conquer O(n log n)	Hybrid (threshold=1000)
1,000	0.1044 (measured, 499,500 ops)	0.0049	0.1059
10,000	9.5302 (measured, 49,995,000 ops)	0.0622	0.0579
100,000	~953.0 (extrapolated, O(n^2))	0.8211	0.8027
1,000,000	~95,302 (~26.5 hours, extrapolated)	11.5876	11.9716



Brute Force was actually executed (not estimated) for n = 1,000 and n = 10,000; beyond that its O(n^2) growth was extrapolated from the measured n=10,000 constant, since a literal run at n=10^6 would take roughly a day of compute for a single query — this is discussed further as a limitation in Section 4. Divide-and-Conquer and Hybrid were both executed for real, for every size up to n = 1,000,000, completing the largest case in under 12 seconds.

Convex Hull — execution time (seconds)

n	Brute Force O(n^3)	QuickHull / D&C	Hybrid (threshold=500)
50 / 100 / 200 / 400 / 800	0.016 / 0.084 / 0.526 / 3.139 / 22.654 (all measured)	-	-
1,000	~49.0 (extrapolated)	0.0022	0.0022
10,000	~49,047 (~13.6 hrs, extrapolated)	0.0227	0.0221
100,000	~4.9x10^7 (~567 days, extrapolated)	0.2047	0.2149
1,000,000	~4.9x10^10 (~1,554 yrs, extrapolated)	3.7590	3.5652



Brute-Force Convex Hull was actually executed (not estimated) for n = 50, 100, 200, 400 and 800; already at n = 800 it needed over 22 seconds, and its O(n^3) growth makes n >= 1,000 impractical to run literally, so those figures are extrapolated from the measured constant at n = 400. QuickHull and Hybrid were both executed for real at every required size, completing n = 1,000,000 in under 4 seconds.

3.6 Complexity Analysis (Part B algorithms)

Algorithm	Time Complexity	Space Complexity
Brute Force Closest Pair	O(n^2)	O(1) extra / O(n) input
Divide & Conquer Closest Pair	O(n log n)	O(n) (recursion + strip arrays)
Hybrid Closest Pair	O(n^2) if n<=threshold, else O(n log n)	O(n)
Brute Force Convex Hull	O(n^3)	O(n)
QuickHull (D&C) Convex Hull	O(n log n) average, O(n^2) worst case	O(n)
Hybrid Convex Hull	O(n^3) if n<=threshold, else avg O(n log n)	O(n)

4. Comparative Analysis — Brute Force vs Divide-and-Conquer vs Hybrid

- Basic strategy: Brute Force checks every possible pair/edge exhaustively with no use of structure in the data; Divide-and-Conquer splits the point set recursively (by x-coordinate for Closest Pair, by farthest-point partitioning for QuickHull) and combines partial solutions, exploiting geometric locality to avoid redundant comparisons; Hybrid simply routes small inputs to Brute Force (where its low constant-factor overhead wins) and large inputs to Divide-and-Conquer (where its better asymptotic growth wins).
- Distance / orientation calculations: Brute Force performs O(n^2) distance calculations (Closest Pair) or O(n^3) orientation tests (Convex Hull); Divide-and-Conquer performs O(n log n) work overall because each recursion level only re-examines a narrow 'strip' of candidate points (Closest Pair) or a shrinking farthest-point subset (QuickHull) rather than the whole remaining set.
- Computational cost as n increases: the experimental results in Section 3.5 show Brute Force cost exploding — from a fraction of a second at n=1,000 to an estimated ~26.5 hours (Closest Pair) or ~1,554 years (Convex Hull) at n=1,000,000 — while Divide-and-Conquer and Hybrid stay under 12 seconds and 4 seconds respectively across the entire range, a difference of many orders of magnitude.
- Limitations for large datasets: pure Brute Force is simply not usable beyond a few thousand (Closest Pair) or a few hundred (Convex Hull) points in practice; it is included here only as the theoretical baseline and for the smaller Part A demonstration where n=10. QuickHull's worst case (all points nearly collinear or already on the hull) degrades to O(n^2), so a hybrid or a guaranteed-O(n log n) alternative (e.g. Graham scan / Andrew's monotone chain, or true Divide-and-Conquer hull merging) is preferable when worst-case guarantees matter.
- Possible improvements: replace Brute Force entirely for n beyond a few hundred/thousand points; for Closest Pair, the classic Divide-and-Conquer algorithm used here already achieves the optimal O(n log n); for Convex Hull, Graham's scan (O(n log n) guaranteed) or Chan's algorithm (O(n log h), output-sensitive in the hull size h) would remove QuickHull's worst-case risk while keeping the same average-case benefit observed here.

5. Limitations of This Study

- Timing was single-run per configuration rather than averaged over multiple repetitions, so measured values include some system-level noise (garbage collection, OS scheduling); repeated trials with mean/variance would strengthen the results.
- Coordinates were synthetically generated within a real-world bounding box rather than sourced from a literal downloaded dataset, due to network access constraints in the execution environment; the geometric behaviour of uniform random points is nevertheless a standard and defensible proxy used throughout the computational-geometry benchmarking literature.
- Pure Brute Force timings beyond the measured range (n=10,000 for Closest Pair; n=800 for Convex Hull) are analytically extrapolated from the O(n^2)/O(n^3) growth law rather than actually executed, since literal execution would take from tens of minutes to well over a thousand years for the largest case.
- QuickHull's O(n^2) worst case (e.g. all points already on the hull, or collinear input) was not specifically stress-tested; uniform random data mostly avoids this pathological case, so the reported timings reflect QuickHull's favourable average case rather than its worst case.
- All experiments ran on a single machine/process without parallelism; a distributed or multi-threaded implementation could shift the practical thresholds further.

6. Final Conclusion

The Brute Force technique provides a simple and reliable way to solve both the **Closest Pair** and **Convex Hull** problems. For the given 10 points, the closest pair is **(9,4) and (7,2)** with a minimum Euclidean distance of approximately **2.828**. The Convex Hull consists of **7 boundary points**, while the remaining 3 points lie inside the hull.

The Closest Pair Brute Force algorithm performs:

$$\boxed{C(n,2) = \frac{n(n-1)}{2}}$$

distance comparisons and has **O(n²)** time complexity. The Brute Force Convex Hull checks every candidate edge against other points and has **O(n³)** time complexity.

Therefore, Brute Force is suitable for small datasets and for validating other algorithms, but it becomes inefficient as the input size increases. **Divide-and-Conquer** and **Hybrid** approaches provide better scalability and are more suitable for large real-world datasets.

Overall Comparison

$$\boxed{\text{Brute Force} < \text{Hybrid} < \text{Divide-and-Conquer}}$$

in general practical scalability, although the exact performance depends on implementation and the chosen Hybrid threshold.